東北大學 国家示范性软件学院
SOFTWARE COLLEGE OF NORTHEASTERN UNIVERSITY

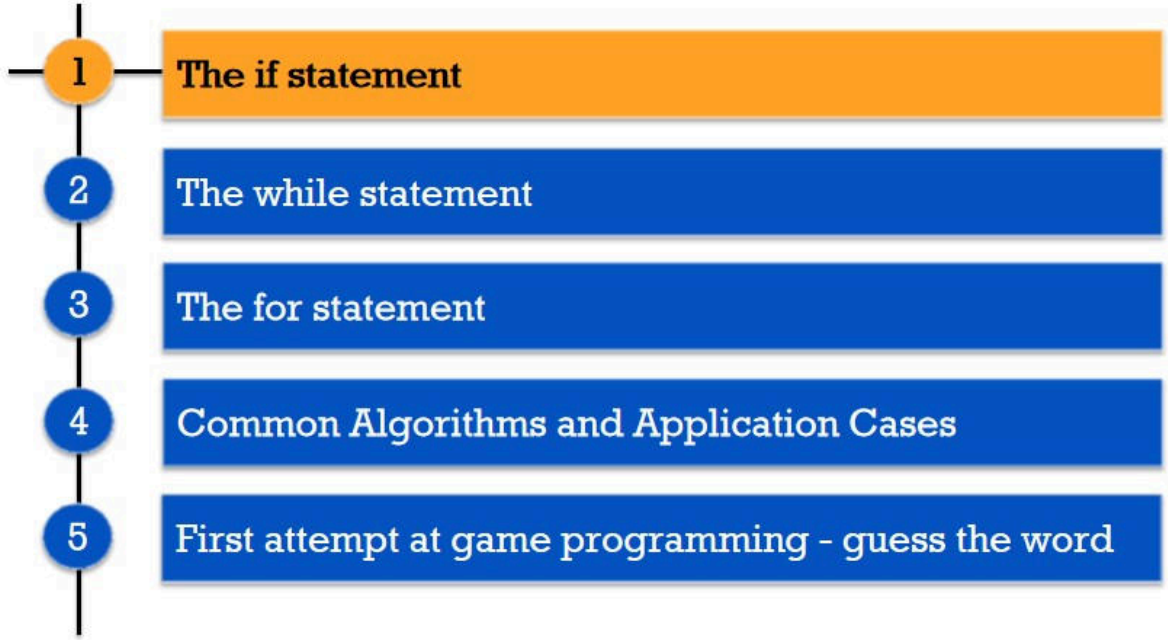


Programming in Python
—— PART I Basics

1

Chapter 3: Control statements

Contents



从C/Java到Python的思维转变

- 在C与Java中，我们早已熟悉了 if、for、while 等控制流的核心逻辑，这是所有编程语言的通用基石。Python不仅继承了这些逻辑，更通过极简的语法设计重构了表达形式，让代码的可读性与书写效率达到新的高度。

01 强制缩进的美学

- 摒弃传统的大括号，用缩进严格界定代码块。这不仅是语法规则，更是强制代码整洁的机制，让可读性成为Python的灵魂标识。

02 遍历优先的循环

- 不再依赖计数器，Python的 for 循环天生用于遍历。无论是列表、字符串还是字典，都能直接迭代，让逻辑更直观、代码更简洁。

03 更优雅的语法表达

- 支持 elif 多分支判断，甚至允许循环后紧跟 else 子句。这些“语法糖”让代码逻辑更贴近自然思维，减少冗余嵌套。

从C/Java到Python的思维转变

- AI时代的进阶思考
  - 大模型可以生成代码，但无法替代逻辑思考
  - 理解控制流的底层逻辑，是驾驭AI工具的前提
  - 我们应该成为代码的“指挥官”，而不是被动的“复制者”

雨课堂 Rain Classroom

条件语句：if-elif-else

功能场景	Python 语法	Java / C 语法	核心差异
简单单向判断	`if cond: ...`	`if(cond){ ... }`	无括号，冒号结尾
双向分支选择	`if cond: ...` `else: ...`	`if(cond){...}else{...}`	缩进区分层级
多向条件分支	`if/elif/else`	`if/else if/else`	关键字更简洁

•缩进是语法核心

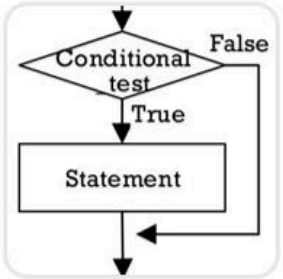
•缩进不是风格，而是强制语法。Python通过缩进识别代码块，而非大括号。

•pass 占位符

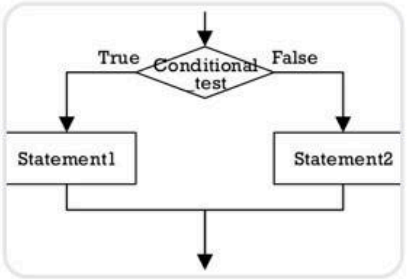
•当逻辑暂未实现但需保持结构完整时，使用 pass 避免语法错误，起到占位作用。

•match 模式匹配

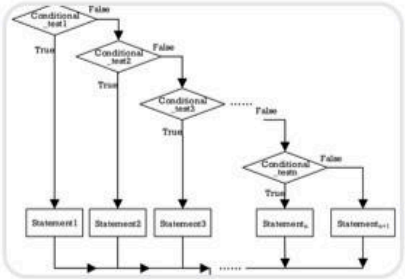
•Python 3.10+ 引入，替代传统 switch，支持复杂的模式匹配与解构赋值，更灵活。



简单判断逻辑



双向分支逻辑



多向分支逻辑

逻辑可视化

流程图清晰展示了程序的执行路径。理解这些逻辑流向，是编写无Bug代码的关键基础。

雨课堂 Rain Classroom

The if statement

1. Simple if Statements

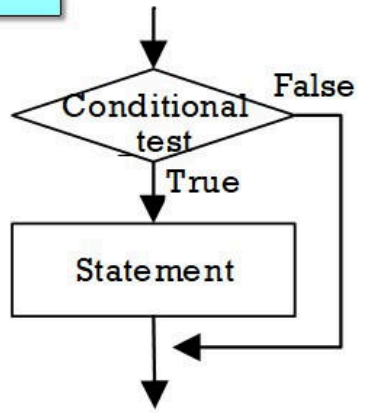
- The simplest kind of if statement has one test and one action:

```
if conditional_test :
    do something
```

- Expressions for Judging Relationships:
  >、<、==、>=、<=
- logical expression:
  and or not

voting.py

```
age = 19
❶ if age >= 18:
❷     print("You are old enough to vote!")
```

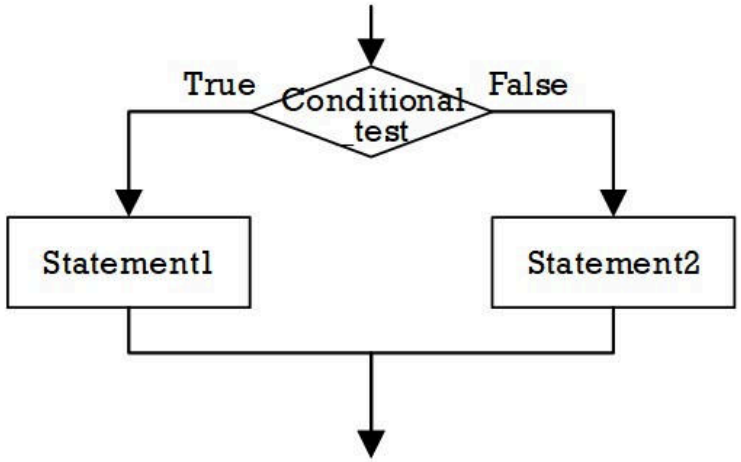


The if statement

2. if-else Statements

- Often, you'll want to take one action when a conditional test passes and a different action in all other cases. Python's if-else syntax makes this possible.

```
if conditional_test:
    statement segment 1
else:
    statement segment 2
```

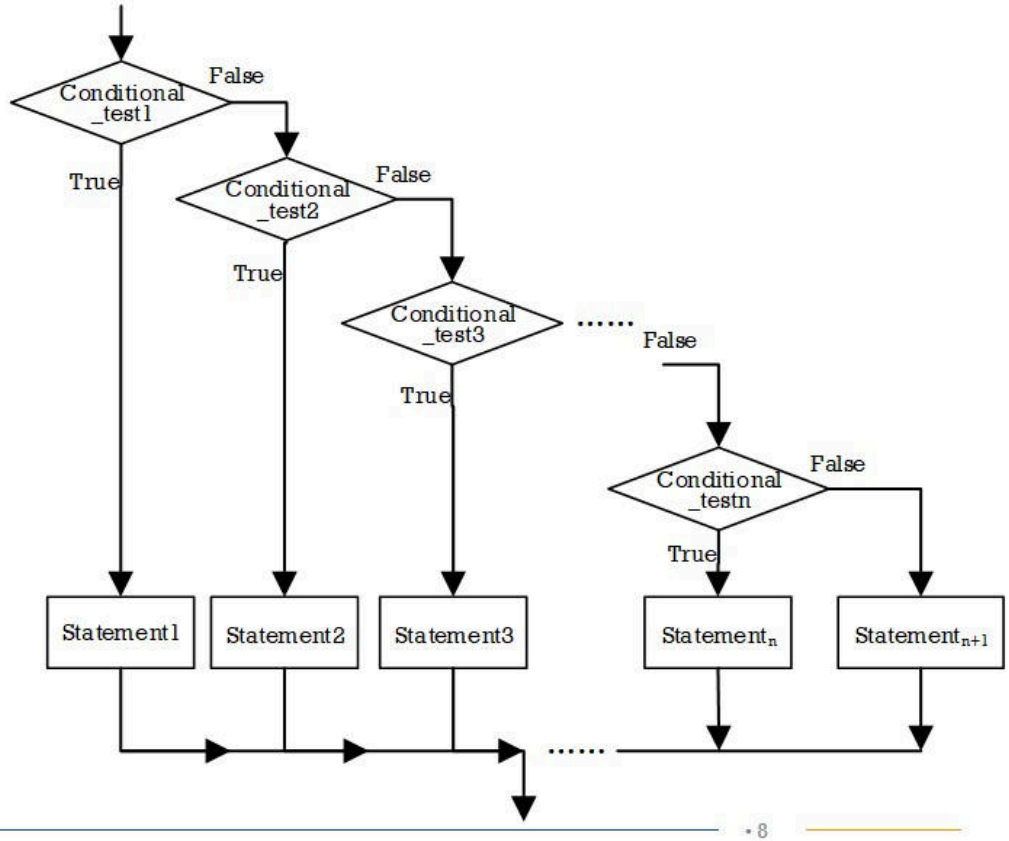


The if statement

3. The if-elif-else Chain

- Python executes only one block in an **if-elif-else** chain.

```
if conditional_test1 :
    statement segment1
elif conditional_test2 :
    statement segment2
    ......
elif conditional_testn :
    statement segmentn
else:
    statement segmentn+1
```



The if statement

Example:

- Converting score in hundred-mark system to five-letter grading system

```
score = input("请输入学生成绩：")
score = int(score)
if score >= 90:
    print ('成绩为优')
elif score >= 80:
    print ('成绩为良')
elif score >= 70:
    print ('成绩为中等')
elif score >= 60:
    print ('成绩为及格')
else:
    print ('成绩为不及格')
```


雨课堂 Rain Classroom

The if statement

4. pass statement

- Similar to Null statements, can be used in class and function definitions, or in branching structures

```
if a < b :          # in branch statement
   pass
else:
   z = a

class A :           # in class definition
      pass

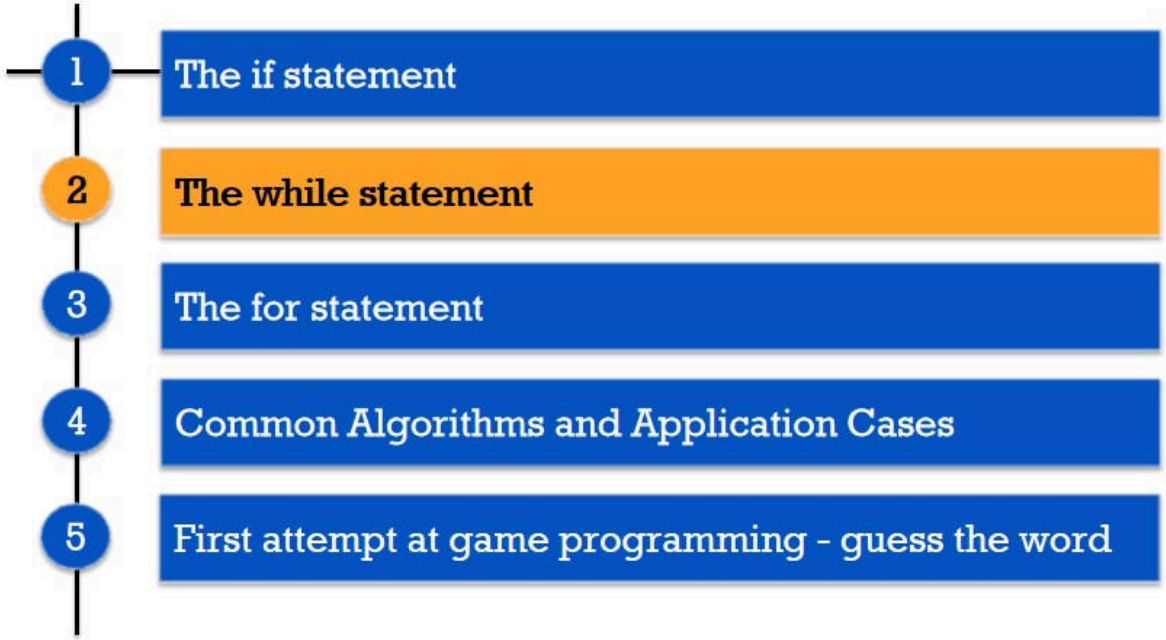

def demo( ) :           # in function definition
      pass
```

- 当暂时没有确定如何实现功能，或者为以后的软件升级预留空间，或者其他类型功能时，使用 “pass” 来占位。

雨课堂 Rain Classroom

Chapter 3: Control statements

Contents



雨课堂 Rain Classroom

The while statement

1. while Loops

- The while loop runs as long as, or *while*, a certain condition is true.
- Python provides ***for loops*** and ***while loops***, no do...while loop

```
while conditional_test :
    do something
```

```
count = 0
while count < 9 :
    print ('The count is:', count)
    count = count + 1
print ("Good bye!" )
```


雨课堂 Rain Classroom

The while statement

2. Letting the User Choose When to Quit

- We'll define a *quit value* and then keep the program running as long as the user has not entered the quit value:

```
prompt = "\nTell me something, and I will repeat it back to you:"
prompt += "\nEnter 'quit' to end the program. "
message = ""
while message != 'quit':
  message = input(prompt)
  print(message)
```


雨课堂 Rain Classroom

The while statement

3. Sometimes, the conditional_test can be a constant

```
count = 0
while 1:
  print ('The count is:', count)
  count = count + 1
print ("Good bye!" )
```

- This is an infinite loop program that can be terminated with the help of a ***break*** statement.

雨课堂 Rain Classroom

The while statement

例题：输入两个正整数，求它们的最大公约数

- 分析：求最大公约数可以使用“辗转相除法”

（1）比较两个数，并使 m > n；

（2）将m作被除数，n作除数，相除后余数为r；

（3）若 r != 0，则 m = n，n = r 继续执行步骤（2），否则，n为最大公约数，结束循环。

雨课堂 Rain Classroom

The while statement

例题：输入两个正整数，求它们的最大公约数

```
num1 = int(input("输入第一个数字: "))
num2 = int(input("输入第二个数字: "))
m = num1
n = num2
if m < n:
  m, n = n, m   # m, n交换值
r = m % n
while  r!=0:
   m = n
   n = r
   r = m % n
print( num1,"和", num2,"的最大公约数为", n)
```

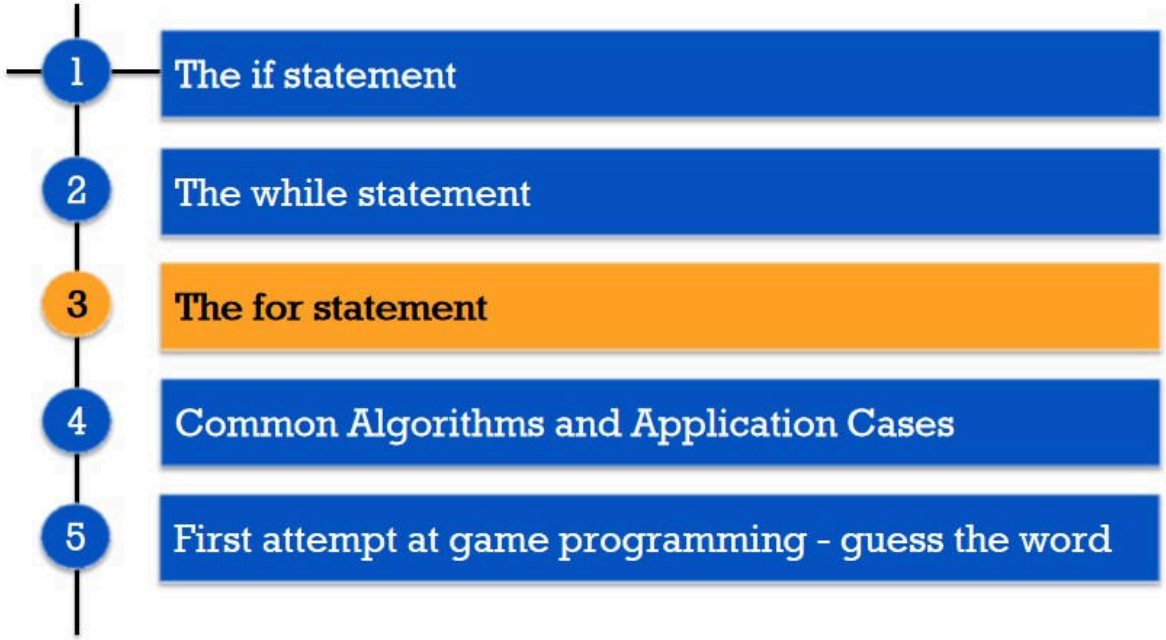
Output Result:

```
输入第一个数字: 36
输入第二个数字: 48
36 和 48 的最大公约数为 12
```


雨课堂 Rain Classroom

Chapter 3: Control statements

Contents



The for statement

- The for statement is used to iterate over the elements of a sequence (such as a string, tuple or list) or other iterable object.

```
for item in Iterable sequence:
    do something
```

- for 循环的本质就是先通过 iter() 函数获取可迭代对象的迭代器；
- 然后对获取到的迭代器不断调用 next() 方法，来获取下一个值并将其赋值给item；
- 当遇到 StopIteration 异常后结束循环。

```
# Example-1:
for letter in 'Python':
    print ( 'Current letter is:', letter )
```

```
>>> for letter in 'Python':
...     print ( 'Current letter is:', letter )
...
Current letter is: P
Current letter is: y
Current letter is: t
Current letter is: h
Current letter is: o
Current letter is: n
>>>
```

补充：可迭代对象

iter(), next()

- 对于可迭代对象，都可以通过iter()函数获取这些可迭代对象的迭代器.
- 然后可以对获取到的迭代器不断的使用next()函数来获取可迭代对象的下一个元素
- 直至没有数据可获取，则产生StopIteration异常

```
>>> list1 = [1, 2, 3, 4, 5]
>>> iter_L1 = iter(list1)
>>> next(iter_L1)
1
>>> next(iter_L1)
2
>>> next(iter_L1)
3
```

```
>>> next(iter_L1)
4
>>> next(iter_L1)
5
>>> next(iter_L1)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
StopIteration
```


雨课堂 Rain Classroom

The for statement

```
# Example-2:  for loop iterates over the elements in the list

fruits = ['banana', 'apple',  'mango']
for fruit in fruits:
      print ( 'element :', fruit)
print( "Good bye!" )


Output Result:
element :  banana
element :  apple
element :  mango
Good bye!
```


雨课堂 Rain Classroom

The for statement

```
# Example-3: Calculate the sum of integers from 1-10
sum = 0
for x in [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]:
    sum = sum + x
print(sum)
```

Output Result: 55

```
# Example-4: Calculate the sum of integers from 1-100
sum = 0
for x in range(1,101):
    sum = sum + x
print(sum)
```

Output Result: 5050

range(start=0, stop[, step])
Python3 Build-in function

```
>>> range(5)
range(0, 5)
>>> for i in range(5):
...     print(i)
...
0
1
2
3
4
```

- range()返回一个可迭代对象，不是列表
- list()函数可以把返回的对象转为一个列表

```
>>> list(range(5))
[0, 1, 2, 3, 4]
```

The for statement

```
# Example-5:  for loop through index

fruits = ['banana', 'apple',  'mango']
for i in range(len(fruits)):
    print( 'Current fruit :', fruits[i] )
print ("Good bye!")
```

Output Result:

```
Current fruit : banana
Current fruit : apple
Current fruit : mango
Good bye!
```

- Build-in function:
  - range()
  - len()

雨课堂 Rain Classroom

Using break to Exit a Loop

- The *break* statement can be used in both *while loops* and *for loops*.
- Generally, it is placed in the if statement. Once the break statement is executed, the entire loop will end immediately.

```
i = 1
while 1:          # infinite loop
   print (i)      # output 1-10
   i += 1
   if i > 10:     # Exit the loop when i is greater than 10
      break
```

■ 除非 break 语句让代码看起来更简单或更清晰，否则不要轻易使用

雨课堂 Rain Classroom

Using continue in a Loop

- The *continue* statement terminates the current loop based on the result of a conditional test, ignores the statement after continue, and returns to the beginning of the loop to advance to the next loop.

```
i = 1
while i < 10:
   i += 1
   if i%2 > 0:   # Skip output if not even
      continue
   print (i, end=', ')

Output Result:
2, 4, 6, 8, 10,
```


雨课堂 Rain Classroom

loop nesting

- The Python language allows a loop to be nested inside another loop.
- For example
  - a *for loop* can be nested in a *while loop*;
  - You can also nest *while loops* within *for loops*.
- The nesting level is generally no more than 3 levels to ensure readability.

雨课堂 Rain Classroom

loop nesting

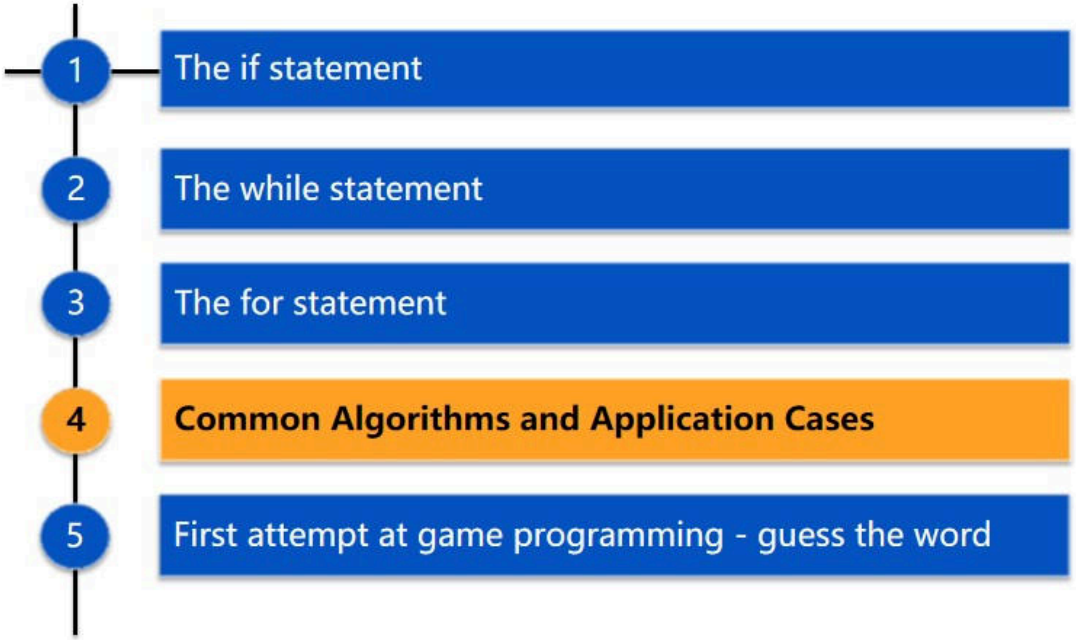
Example: print the nine-nine multiplication table

```
for i in range(1, 10):
    for j in range(1, i+1):
        print(i, '*', j, '=', i*j, '\t', end = '')
    print('')
```

```
1 * 1 = 1
2 * 1 = 2       2 * 2 = 4
3 * 1 = 3       3 * 2 = 6       3 * 3 = 9
4 * 1 = 4       4 * 2 = 8       4 * 3 = 12      4 * 4 = 16
5 * 1 = 5       5 * 2 = 10      5 * 3 = 15      5 * 4 = 20      5 * 5 = 25
6 * 1 = 6       6 * 2 = 12      6 * 3 = 18      6 * 4 = 24      6 * 5 = 30      6 * 6 = 36
7 * 1 = 7       7 * 2 = 14      7 * 3 = 21      7 * 4 = 28      7 * 5 = 35      7 * 6 = 42      7 * 7 = 49
8 * 1 = 8       8 * 2 = 16      8 * 3 = 24      8 * 4 = 32      8 * 5 = 40      8 * 6 = 48      8 * 7 = 56      8 * 8 = 64
9 * 1 = 9       9 * 2 = 18      9 * 3 = 27      9 * 4 = 36      9 * 5 = 45      9 * 6 = 54      9 * 7 = 63      9 * 8 = 72      9 * 9 = 81
```

Chapter 3: Control statements

Contents



Common Algorithms and Application Cases

Example-1: Find the approximate value of the *natural logarithm e* (累乘/累加算法)

➢ **Analysis:**

- The approximate formula is: $e = 1 + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \cdots + \frac{1}{n!}$

  - 累乘(n!)、累加
  - 在原有数据基础上不断加/乘

- This is a convergent series(收敛级数) that can be approximated by summing its first *n* terms.
- But where the calculation is, that is, how much is n to end the calculation.
- This involves a calculation error(计算误差) problem. We can set the calculation to stop when the value of the *nth* item is less than 10^{-5}.

雨课堂 Rain Classroom

Common Algorithms and Application Cases

$$e = 1 + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \cdots + \frac{1}{n!}$$

```
i = 1
p = 1
sum_e = 1
t = 1/p
while t > 0.00001 :          # 误差判断
    p = p * i                # 计算 i 的阶乘
    t = 1 / p                # 计算 第 i 项，也是误差
    sum_e = sum_e + t        # 累加
    i += 1
print("自然对数e的近似值" , sum_e)
```

Output Result:

自然对数e的近似值 2.7182815255731922

雨课堂 Rain Classroom

Common Algorithms and Application Cases

Example-2: Find the largest of 10 *random integers* in the interval [100, 200] (打擂算法)

- **Analysis:**
  - The algorithms for finding the largest and smallest numbers in a set of data are similar, and the "beating" algorithm can be used
  - Example for finding the largest number, first take the first number as the largest number, and then compare it with all other numbers one by one to find the largest number to replace the first number, and then take the second number as the second largest number, and so on .....

    - Importing the random module, where ***random.randomrange()*** can get a random number from the specified range

雨课堂 Rain Classroom

Common Algorithms and Application Cases

```
import random
x = random.randrange(100, 201)          # 产生一个[100, 200]之间的随机数 x
maxn = x                                # 设定最大数
print(x, end = " ")                     # 输出不换行
for i  in range(2, 11):
    x = random. randrange(100, 201)  # 再产生一个[100, 200]之间的随机数 x
    print(x, end = " ")
    if x > maxn :
        maxn = x;                       # 若新产生的随机数大于最大数，则打擂成功，变新擂主
print ("\n最大数：  ", maxn)

Output Result:
138 119 125 142 116 190 125 159 128 169
最大数：  190
```

Common Algorithms and Application Cases

```
import random
a = [ ]                                  # Create an empty list
for i  in range(1, 11):
    x = random. randrange(100, 201)
    print(x, end = " ")
    a.append(x)                          # 将生成的随机数放入列表中
print ("\n最大数:  ", max(a))
```

Output Result:

132 118 153 107 124 141 151 143 112 194

最大数: 194

雨课堂 Rain Classroom

Common Algorithms and Application Cases

Example-3: Buy 100 chickens for 100 yuan (enumeration 枚举算法)

- 公鸡每只5元，母鸡每只3元，小鸡3只1元

```
for x in range(0, 100):
    for y in range(0, 100):
        z = 100 - x - y
        if z >= 0 and 5*x + 3*y + z/3 == 100 :
            print ('公鸡%d只，母鸡%d只，小鸡%d只' % (x, y, z))
```

Output Result:

公鸡0只，母鸡25只，小鸡75只
公鸡4只，母鸡18只，小鸡78只
公鸡8只，母鸡11只，小鸡81只
公鸡12只，母鸡4只，小鸡84只

雨课堂 Rain Classroom

Common Algorithms and Application Cases

Example-4: Output the first 20 terms of the Fibonacci sequence (递推算法)

```
f1=1
f2=1
print("(1): ",f1)
print("(2): ",f2)
for i in range(3,21):
    f3 = f1 + f2                # 递推公式
    print (f'({i}): {f3}')
    f1 = f2
    f2 = f3
```

Output Result:

(1): 1
(2): 1
(3): 2
(4): 3
(5): 5
(6): 8
(7): 13
(8): 21
(9): 34
(10): 55
(11): 89
(12): 144
(13): 233
(14): 377
(15): 610
(16): 987
(17): 1597
(18): 2584
(19): 4181
(20): 6765

- 利用递推算法或迭代算法，可以将一个复杂的问题转换为一个简单过程的重复执行。
- 这两种算法的共同特点是，通过前一项的计算结果推出后一项。
- 不同的是，递推算法不存在变量的自我更迭，而迭代算法则在每次循环中用变量的新值取代其原值

Common Algorithms and Application Cases

Example-5: Find the square root of the variable *a* (迭代算法)

- **Analysis:**
  - Calculation formula for square root using iterative algorithm: $x_{n+1} = \frac{\left(x_n + \frac{a}{x_n}\right)}{2}$

- **Algorithm step**

  (1) Set an initial value x_0 (eg $x_0 = a/2$)

  (2) Use the formula to calculate x_1, then x_1 must be very different from the real square root.

  (3) If the error is greater than 10^{-5}, use x_1 as x_0, and re-calculate x_1 until the error is less than 10^{-5}

Common Algorithms and Application Cases

Example-5: Find the square root of the variable a (迭代算法)

```
a=int(input("Input a positive number: "))      # Input a positive number
x0 = a / 2;                                    # take an initial value
x1 = (x0 + a / x0) / 2                         #
while abs(x1-x0) > 0.00001:                    # Determine whether the error is less than 10-5
   x0 = x1
   x1 = (x0 + a / x0) / 2
print("The square root is: ", x0)
```

Output Result:
Input a positive number: 2
The square root is: 1.4142137800471977

$$x_{n+1} = \frac{\left(x_n + \frac{a}{x_n}\right)}{2}$$

多选题 1分

以下哪些语句在Python中用于流程控制?

A if语句

B while语句

C for语句

D switch语句

雨课堂 Rain Classroom

多选题 2分

关于简单if语句的描述，以下哪项是正确的?

A if语句后必须跟一个else语句

B if语句后面可以有多个elif语句

C if语句至少需要一个elif和一个else

D if语句可以只有一个测试条件和相应的动作

雨课堂 Rain Classroom

多选题 1分

以下哪些语句属于Python中的控制流语句?

A if

B while

C for

D print

雨课堂 Rain Classroom

单选题 1分

以下关于while循环的描述正确的是?

A while循环至少执行一次循环体

B while循环的条件测试在循环体的开始处

C while循环使用break语句跳出循环

D 以上选项都正确

多选题 1分

以下哪些函数可以用来处理可迭代对象?

A iter()

B next()

C sum()

D len()

雨课堂 Rain Classroom

单选题 1分

在Python中，以下哪个语句可以用来立即退出循环?

A return

B break

C continue

D pass

雨课堂 Rain Classroom

单选题 1分

以下哪个选项是正确的if语句结构?

A if (条件): { 语句 }

B if (条件): { 语句 } else: { 语句 }

C if (条件): { 语句 } elif (条件): { 语句 }

D 以上皆正确

多选题 1分

以下哪些方法可以在Python中生成随机数?

A random.random()

B random.randint()

C random.randrange()

D random.choice()

雨课堂 Rain Classroom

多选题 2分

以下哪个方法可以生成一个指定范围内的随机整数并添加到列表中?

A random.randint(100, 200) then a.append(x)

B random.randrange(100, 200) then a.insert(x)

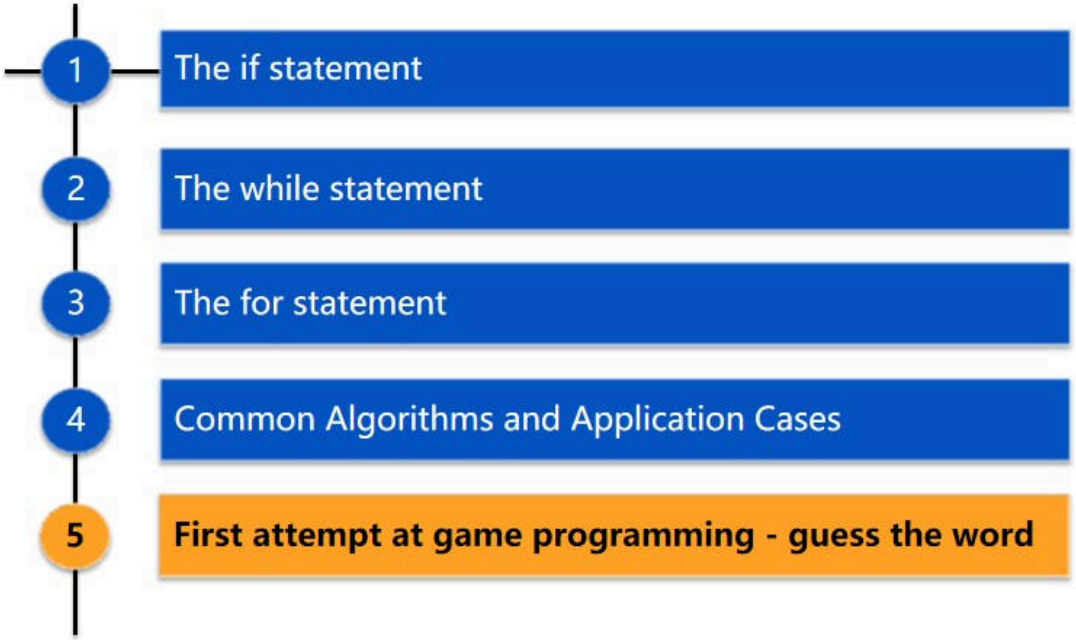
C random.randrange(100, 200) then a.append(x)

D random.randint(100, 200) then a.add(x)

雨课堂 Rain Classroom

Chapter 3: Control statements

Contents



Guess the Word

- **Game description:**
  - The computer randomly selects a word from a collection and shuffles it into alphabetical order, and forms a new word for the player to guess.
- **Analysis:**
  - Where do the words come from?
    - define a tuple containing multiple words

```
WORDS = ("python", "jumble", "easy",
         "difficult", "answer", "continue",
         "phone", "position", "position",
         "game")
```

  - How to randomly select words?
    - Use the function **random.choice()** in the random module

```
欢迎参加猜单词游戏
把字母组合成一个正确的单词.
乱序后单词: yaes
请你猜: easy
真棒，你猜对了!
是否继续（Y/N）: y
乱序后单词: diufctlfi
请你猜: difficutl
对不起不正确.
继续猜: difficult
真棒，你猜对了!
是否继续（Y/N）: n
```

Guess the Word

- 单词字母顺序打乱：
  - 随机从原单词中选择一个位置，提取这个位置的字母放到新单词最后，同时删除原位置的字母，循环多次，直到原单词被抽取为空。

```
while word:      # word不是空串时循环
    position = random.randrange(len(word))      # 根据单词长度，产生单词的随机位置
    jumble += word[position]                    # 将position位置字母组合到乱序后的单词jumble
    word = word[:position] + word[(position + 1):]          # 通过切片，删除原位置字母
```

  - 另外一种方法：

```
import random
l = list(word)          # 将字符串转换成列表
random.shuffle(l)       # 调用random模块的shuffle函数
jumble = ''.join(l)     # 列表转字符串
```

Guess the Word

complete program

```
import random
#创建单词序列
WORDS = ("python", "jumble", "easy", "difficult", "answer", "continue",
                "phone", "position", "position", "game")
# start the game
print(
"""
    欢迎参加猜单词游戏
  把字母组合成一个正确的单词.
"""
)
```


雨课堂 Rain Classroom

Guess the Word

```
iscontinue="y"
while iscontinue == "y" or iscontinue == "Y":
    word = random.choice(WORDS)                          # 从序列中随机挑出一个单词
    correct = word                                       # 一个用于判断玩家是否猜对的变量
    jumble =""                                           # 创建乱序后单词
    while word:                                          # word不是空串时循环
        position = random.randrange(len(word))           # 根据word长度，产生word的随机位置
        jumble += word[position]                         # 将position位置字母组合到乱序后单词
        word = word[:position] + word[(position + 1):]   # 通过切片，删除原位置字母
    print("乱序后单词:", jumble)
    guess = input("\n请你猜: ")
    while guess != correct and guess != "":
        print("对不起不正确.")
        guess = input("继续猜: ")
    if guess == correct:
        print("真棒，你猜对了!\n")
    iscontinue=input("\n\n是否继续（Y/N）: ")
```


雨课堂 Rain Classroom

多选题 1分

以下哪些方法可以用来打乱单词字母顺序?

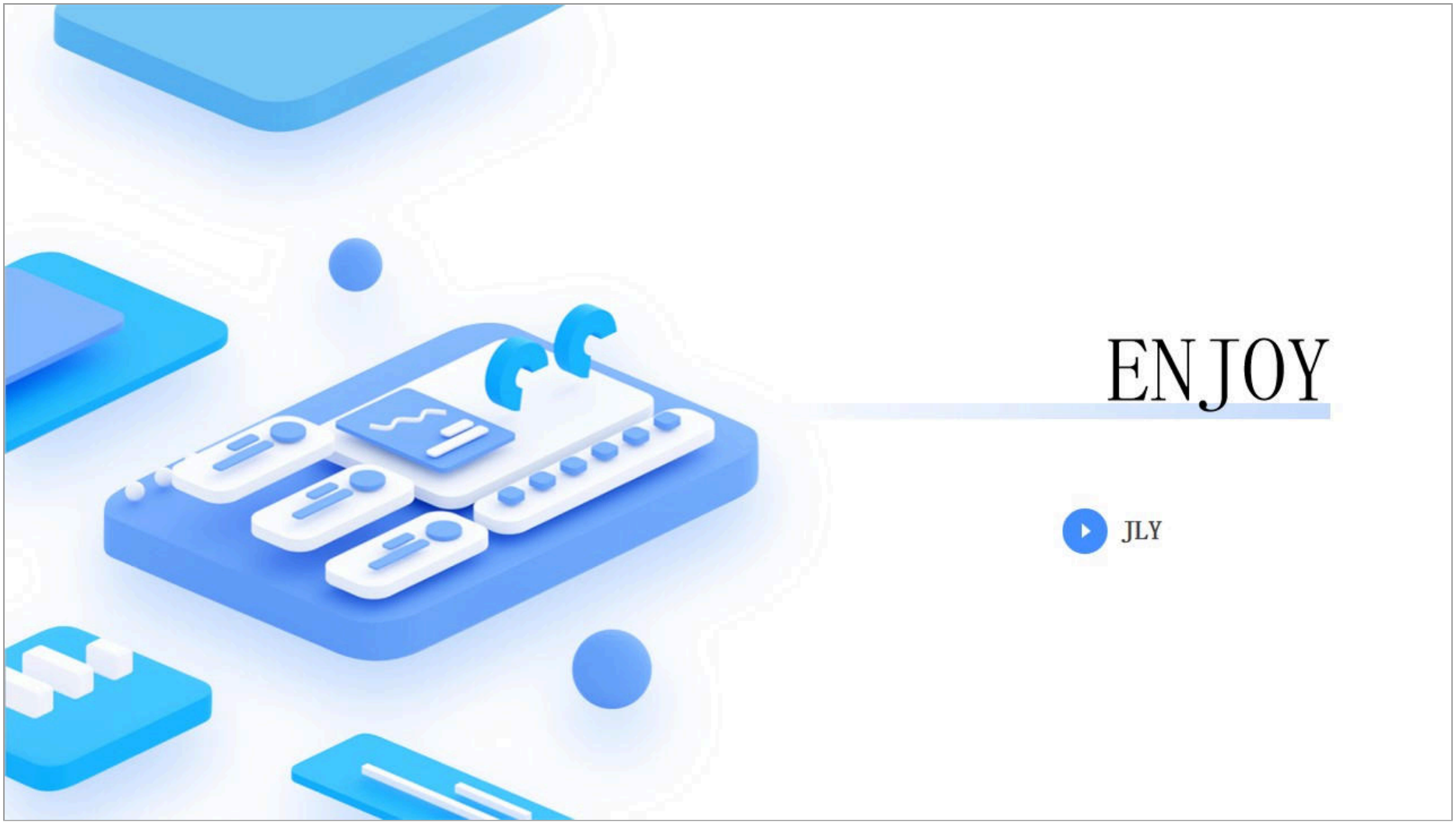
A 随机选择位置，提取字母放到新单词最后

B 使用random.shuffle()函数

C 使用字符串的sort()方法

D 使用列表的reverse()方法

雨课堂 Rain Classroom


ENJOY
JLY